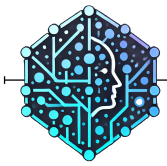


# LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

## A.K.A. AI APPLICATIONS IN PRACTICE

**6 - Numpy**

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

## *NUMPY*

# Numpy

NumPy offers mathematical functions, random number generators, linear algebra routines, and more:

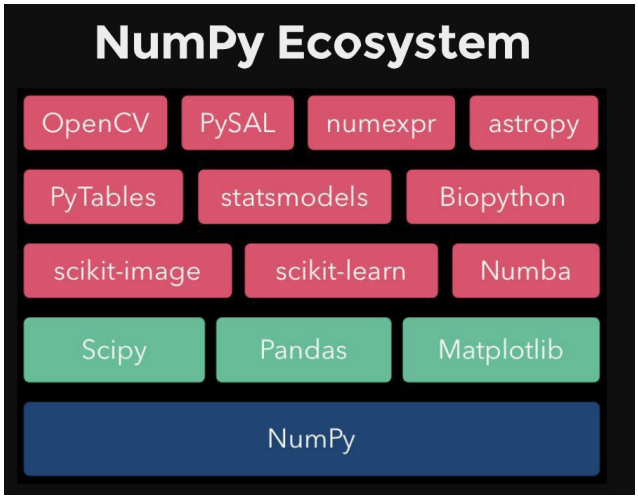
- "de-facto" standards of array computing
- 1D (vector) and 2D arrays (matrix)

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \in \mathbb{R}^n \qquad \mathbf{A} = \begin{bmatrix} a_{11} & \dots & a_{1m} \\ \vdots & & \vdots \\ a_{n1} & & a_{nm} \end{bmatrix} \in \mathbb{R}^{n \times m}$$



# Numpy

Numpy is a general purpose array/matrix manipulation package.



# Numpy

Numpy's object is the homogeneous multidimensional array (ndarray).

It is a table of elements (usually numbers), **all of the same type**, indexed by a tuple of non-negative integers.

In NumPy dimensions are called *axes*.

Example: The (x,y,z) coordinates of a point in a 3-dimensional space is a numpy array with one axis.



# Numpy

A numpy array is created using the "array()" method.

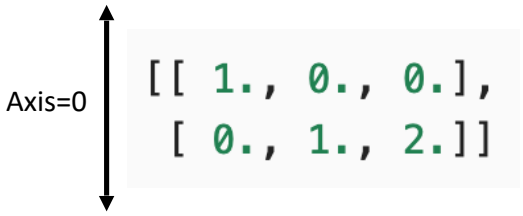
To create an ndarray, we can pass a list, tuple or any array-like object into the array() method, and it will be converted into an ndarray:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
```

# Numpy

General multidimensional arrays have arbitrary numbers of axes:

2 dimensional array:

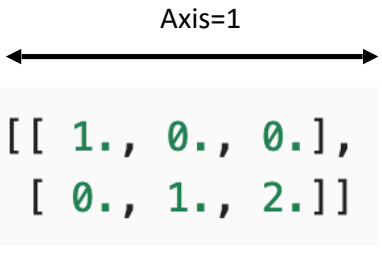





# Numpy

General multidimensional arrays have arbitrary numbers of axes:

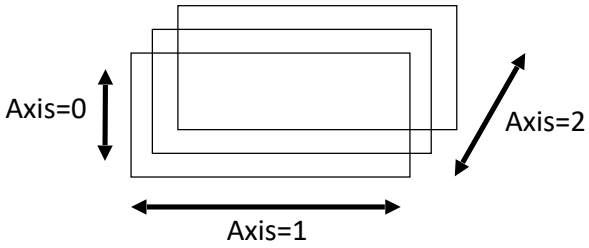
2 dimensional array:



# Numpy

General multidimensional arrays have arbitrary numbers of axes:

3 dimensional array?






# Numpy

Creating multidimensional ndarrays:

```
import numpy as np

arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
arr3 = np.array([[[1, 2, 3, 4], [5, 6, 7, 8]], [[1, 2, 3, 4], [5, 6, 7, 8]]])
```

# Indexing

Given a multidim. array, one can index into the array in the same way as done with lists/string/tuples.

Depending on the number of dimensions (axes) the behavior changes.

One dimensional case is the same as we have already seen.

```
import numpy as np
b = np.array([1,2,3])
b[1]
```

```
2
```



# Indexing

Given a multidim. array, one can index into the array in the same way as done with lists/string/tuples.

In case of more dimensions…

```
import numpy as np
a = np.array([[1,5,4], [3,3,3], [4,4,4]] )
a[0]
```

```
array([1, 5, 4])
```

# Indexing

Given a multidim. array, one can index into the array in the same way as done with lists/string/tuples.

In case of more dimensions…

```
import numpy as np
a = np.array([[1,5,4], [3,3,3], [4,4,4]] )
a[0]
```

```
array([1, 5, 4])
```

If only the first dimension is specified,
all the elements are considered by default



# Indexing

Given a multidim. array, one can index into the array in the same way as done with lists/string/tuples.

In case of more dimensions…

```
import numpy as np
a = np.array([[1,5,4], [3,3,3], [4,4,4]] )
a[0, 2]
```

```
4
```

Comma separated

# Indexing

Given a multidim. array, one can index into the array in the same way as done with lists/string/tuples.

In case of more dimensions…

```
import numpy as np
a = np.array([[1,5,4], [3,3,3], [4,4,4]] )
a[:, 2]
```

```
array([4, 3, 4])
```



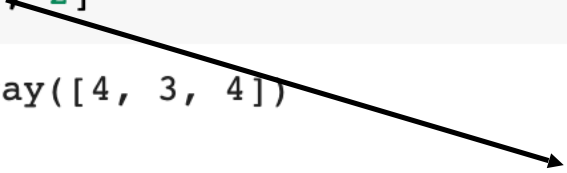
# Indexing

Given a multidim. array, one can index into the array in the same way as done with lists/string/tuples.

In case of more dimensions…

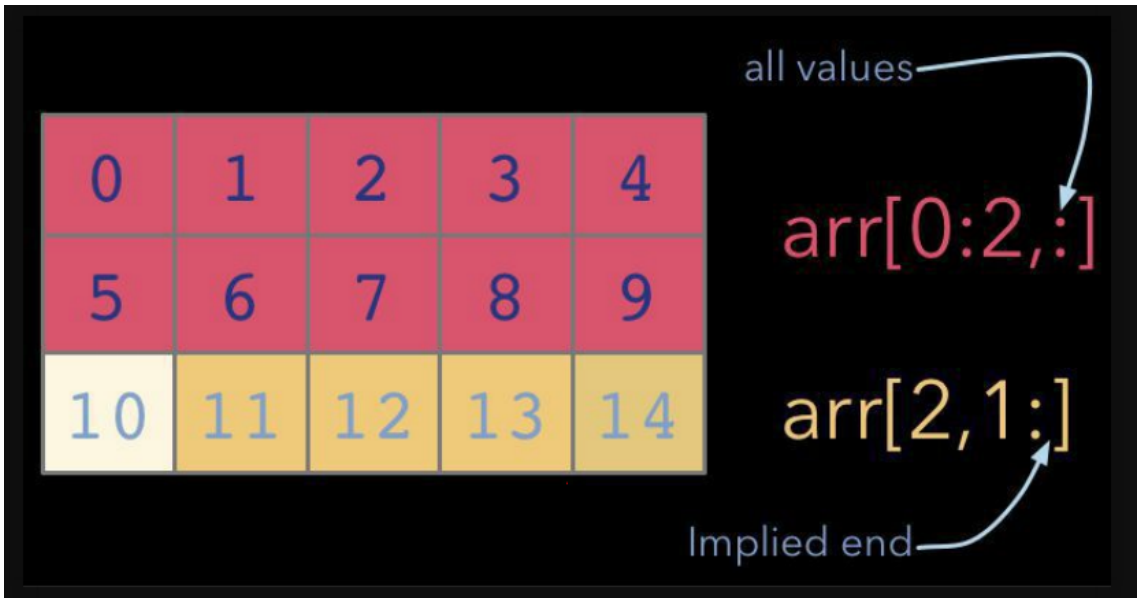
```
import numpy as np
a = np.array([[1,5,4], [3,3,3], [4,4,4]] )
a[:, 2]
```

```
array([4, 3, 4])
```



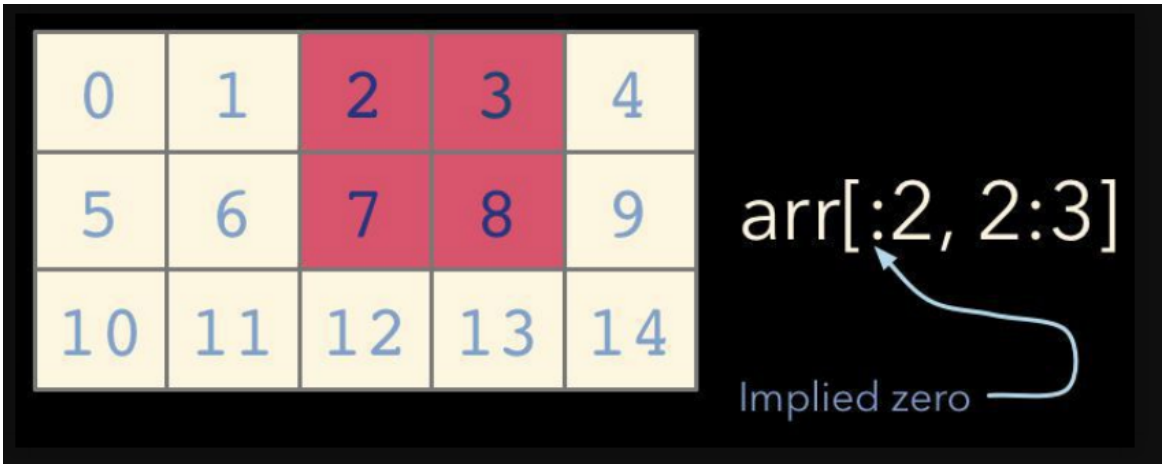
Previous dimensions need to be specified.

# Numpy - slicing

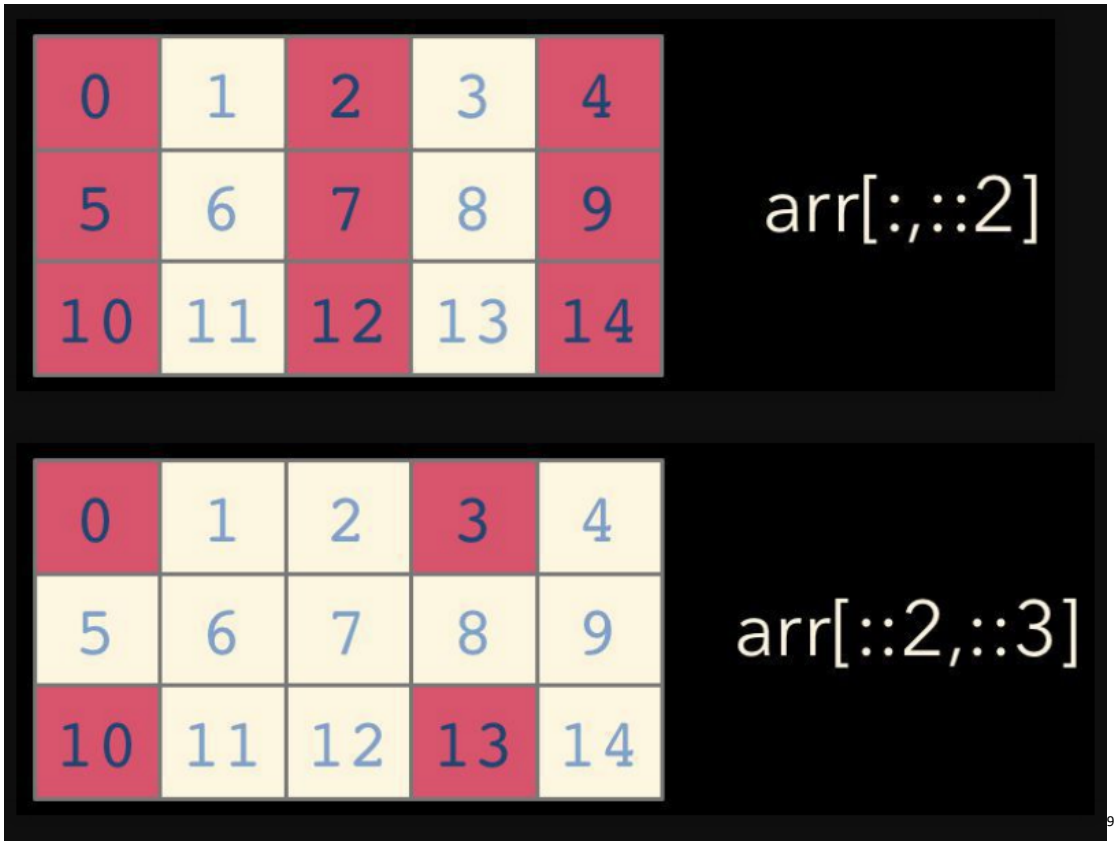





# Numpy - slicing



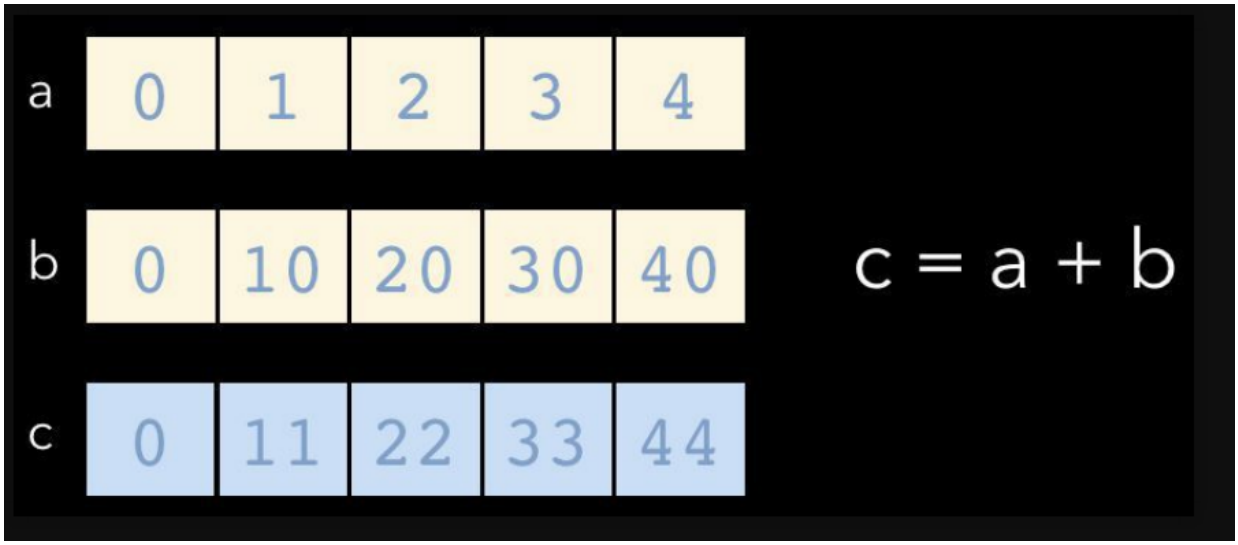
# Numpy - slicing






# Numpy – universal functions

Universal functions in Numpy are functions that are applied element-wise



# Numpy – universal functions

- comparison: `<, <=, ==, !=, >=, >`
- arithmetic: `+, -, *, /, reciprocal, square`
- exponential: `exp, expm1, exp2, log, log10, log1p, log2, power, sqrt`
- trigonometric: `sin, cos, tan, acsin, arccos, atctan`
- hyperbolic: `sinh, cosh, tanh, acsinh, arccosh, atctanh`
- bitwise operations: `&, |, ~, ^, left_shift, right_shift`
- logical operations: `and, logical_xor, not, or`
- predicates: `isfinite, isinf, isnan, signbit`
- other: `abs, ceil, floor, mod, modf, round, sinc, sign, trunc`

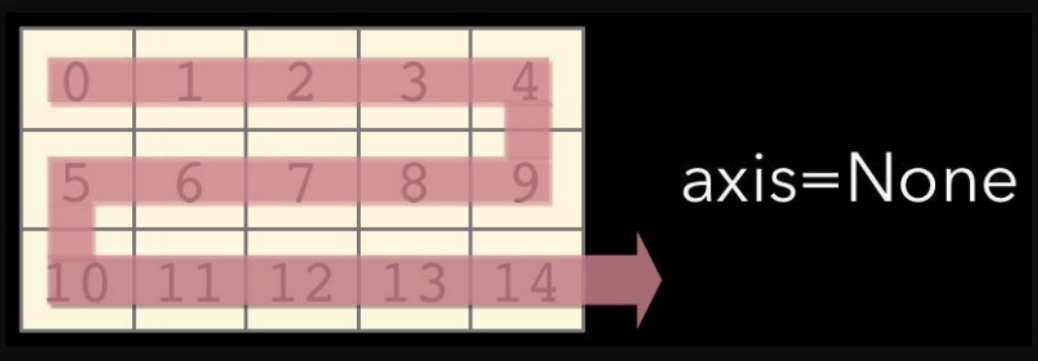


# Numpy - axis

With most functions that operate on tensors in Numpy you can choose on which axis to work on.

The axis is the dimension across which the operation is performed.

```
In [7]: a.sum
()
Out[7]: 105
```

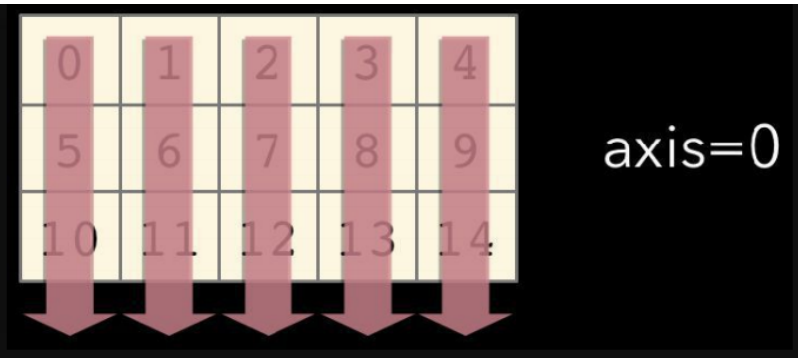


# Numpy - axis

With most functions that operate on tensors in Numpy you can choose on which axis to work on.

The axis is the dimension across which the operation is performed.

```
In [8]: a.sum(axis=0)
Out[8]: array([15, 18, 21, 24,
27])
```



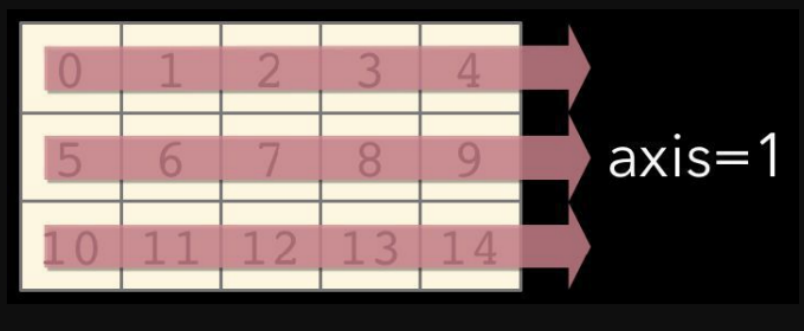



# Numpy - axis

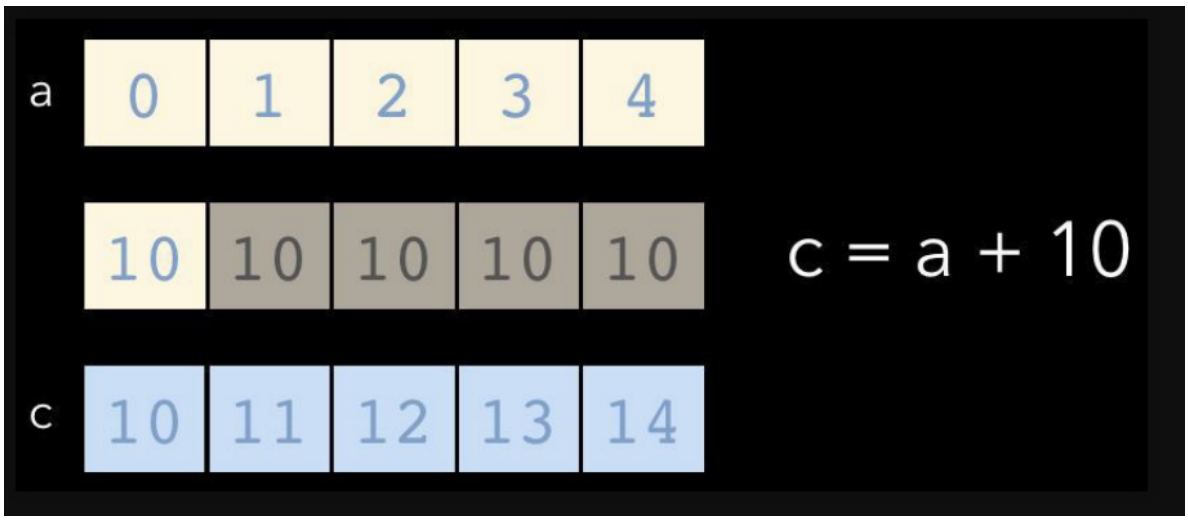
With most functions that operate on tensors in Numpy you can choose on which axis to work on.

The axis is the dimension across which the operation is performed.

```
In [9]: a.sum(axis=1)
Out[9]: array([10,35,60])
```

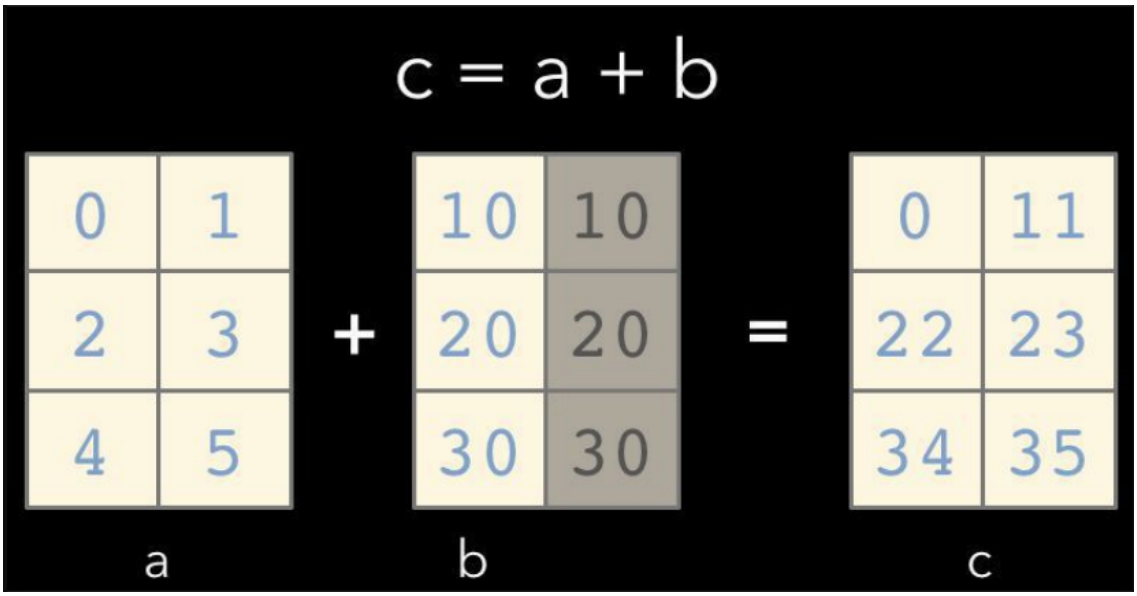


# Numpy - broadcasting






# Numpy - broadcasting



# Numpy – Shapes

NumPy arrays have an attribute called shape that returns a tuple with each index having the number of corresponding elements.

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

print(arr.shape)
```

```
(2, 4)
```



# Numpy – Shapes

Sometimes it can be needed to change the shape of the nd-array.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print(arr)
newarr = arr.reshape(2, 3, 2)
print(newarr)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
[[[ 1  2]
  [ 3  4]
  [ 5  6]]

 [[ 7  8]
  [ 9 10]
  [11 12]]]
```

# Numpy – broadcasting rules

1. Arrays are compatible when dimensions match or one is 1
2. Smaller arrays are "stretched" to match larger ones
3. Missing dimensions are added to the left

```
# Examples of compatible shapes for broadcasting
shapes = [
    ((3,), (3,)),       # Same shapes
    ((3, 1), (1, 3)),   # Both dimensions are compatible
    ((3, 1), (3, 3)),   # Second dimension broadcasted
    ((1, 3), (3, 3)),   # First dimension broadcasted
    ((3, 3), (3,)),     # Missing dimension added to vector
]
```



# Numpy – Exercises

- Create a 1D array with values [1, 2, 3, 4, 5] and a 2D array with shape (5, 1) containing the same values. Perform addition between these arrays and explain the result using broadcasting rules.

- Given a 3x4 array of random values as your choice, reshape it to:
  - a) A 2x6 array
  - b) A 12-element 1D array
  - c) A 4x3 array

# Numpy – flattening

We can also flatten an ndarray, i.e. we can reshape to a vector any n-dimensional array.

The *flatten()* function creates a new array with only a single dimension.

The *ravel()* function, instead, returns a view without creating a new array.

```
# Create a 2D array
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])

# Flatten creates a copy
flattened = arr_2d.flatten()
print(flattened)  # [1 2 3 4 5 6]

# Ravel returns a view when possible
raveled = arr_2d.ravel()
print(raveled)    # [1 2 3 4 5 6]

# Demonstrating view behavior with ravel
raveled[0] = 99
print(arr_2d)     # [[99  2  3], [ 4  5  6]]
```



# Numpy Array vs. Python List

Numpy arrays are similar to Python lists at first glance.

We saw that both can be used as containers with fast item getting and setting. While they are slower for inserts and removals of elements.

For arithmetic, numpy arrays are easily simpler

```
In [3]: a = [1, 2, 3]
        [q*2 for q in a]
Out[3]: [2, 4, 6]
```

```
In [4]: a = np.array([1, 2, 3])
        a * 2
Out[4]: array([2, 4, 6])
```

```
In [1]: a = [1, 2, 3]
        b = [4, 5, 6]
        [q+r for q, r in zip(a, b)]
Out[1]: [5, 7, 9]
```

```
In [2]: a = np.array([1, 2, 3])
        b = np.array([4, 5, 6])
        a + b
Out[2]: array([5, 7, 9])
```

# Numpy Array vs. Python List

Numpy arrays are similar to Python lists at first glance.

We saw that both can be used as containers with fast item getting and setting. While they are slower for inserts and removals of elements.

For arithmetic, numpy arrays are easily simpler

```
In [3]: a = [1, 2, 3]
        [q*2 for q in a]
Out[3]: [2, 4, 6]
```

```
In [4]: a = np.array([1, 2, 3])
        a * 2
Out[4]: array([2, 4, 6])
```

```
In [1]: a = [1, 2, 3]
        b = [4, 5, 6]
        [q+r for q, r in zip(a, b)]
Out[1]: [5, 7, 9]
```

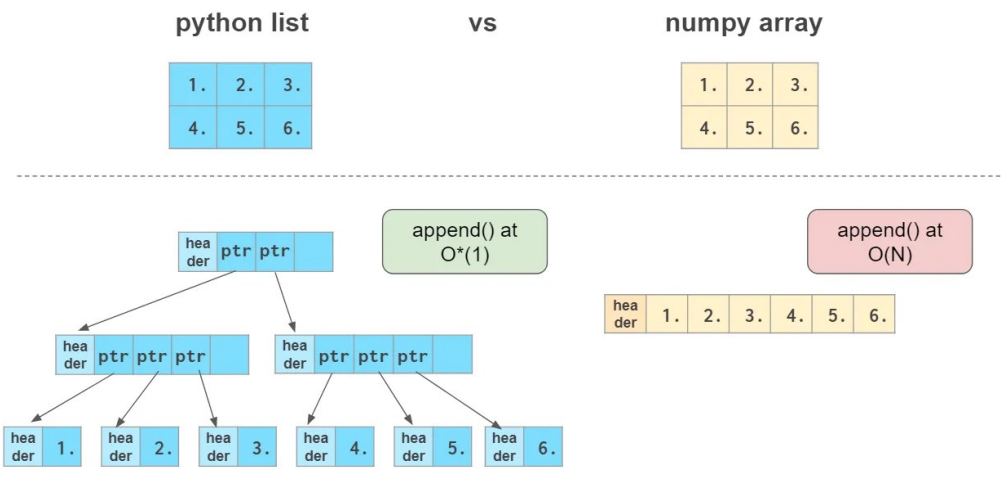
```
In [2]: a = np.array([1, 2, 3])
        b = np.array([4, 5, 6])
        a + b
Out[2]: array([5, 7, 9])
```



# Numpy Array vs. Python List

Numpy arrays are:

- more compact, especially when there's more than one dimension
- faster than lists when the **operation can be vectorized**
- slower than lists when you **append elements** to the end
- they can only work fast with elements of one type



# Numpy – Iterating

Nd-arrays are built from array-like structures e.g. lists.

They are iterable objects and can be used similarly:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

for i, e in enumerate(arr):
  print('Index {} - Element {}'.format(i,e))
```

```
Index 0 - Element 1
Index 1 - Element 2
Index 2 - Element 3
Index 3 - Element 4
Index 4 - Element 5
Index 5 - Element 6
```



# Numpy – Iterating

With more dimensions:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for i,x in enumerate(arr):
  for j,y in enumerate(x):
    print('Axis {} - Index {} - Element {}'.format(i,j,y))
```

```
Axis 0 - Index 0 - Element 1
Axis 0 - Index 1 - Element 2
Axis 0 - Index 2 - Element 3
Axis 1 - Index 0 - Element 4
Axis 1 - Index 1 - Element 5
Axis 1 - Index 2 - Element 6
```

# Numpy – conditional operators

An ndarray can also be indexed by using a binary mask which specifies which indexes to take. Useful with comparison operators.

```
m = np.array([20, -5, 30, 40])
```

```
m<25
```

```
array([ True,  True, False, False])
```

```
m[m<25]
```

```
array([20, -5])
```



# Numpy – fancy indexing

```
# Create an array
arr = np.arange(10)

# Select specific indices
indices = [1, 3, 5, 7]
selected = arr[indices]
print(selected)  # [1 3 5 7]

# 2D example
arr_2d = np.arange(16).reshape(4, 4)
print(arr_2d)
"""
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
"""

# Select specific rows and columns
rows = [0, 2, 3]
cols = [1, 2]
print(arr_2d[rows][:, cols])
"""
[[ 1  2]
 [ 9 10]
 [13 14]]
"""
```

An alternative is to use directly an array containing the integer indexes that we want.

# Views

All the indexing methods except fancy indexing are actually so-called “views”.

They don’t store the data and reflect the changes in the original array if it happens to get changed after being indexed.

All indexing methods, including fancy indexing are mutable: they allow modification of the original array contents through assignment.

python list vs numpy array

```
a = [1, 2, 3]
b = a         # no copy
c = a[:]      # copy
d = a.copy() # copy
```

```
a = np.array([1, 2, 3])
b = a         # no copy
c = a[:]      # no copy!!!
d = a.copy() # copy
```

Credits Lev Maximov



# Numpy – Zeros and ones

Numpy provides functions to create arrays filled with *zeros* or *ones*.
These are useful for initializing arrays before filling them with actual values or creating arrays with specific shapes.

```
# Create a one-dimensional array of zeros
zeros_1d = np.zeros(5)
print("1D array of zeros:")
print(zeros_1d)  # Output: [0. 0. 0. 0. 0.]

# Create a two-dimensional array of zeros
zeros_2d = np.zeros((3, 4))
print("\n2D array of zeros:")
print(zeros_2d)
"""
Output:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
"""
```

# Numpy – Zeros and ones

```
# Create a three-dimensional array of ones
ones_3d = np.ones((2, 3, 2))
print("\n3D array of ones:")
print(ones_3d)
"""
Output:
[[[1. 1.]
  [1. 1.]
  [1. 1.]]

 [[1. 1.]
  [1. 1.]
  [1. 1.]]]
"""

# You can also specify the data type
zeros_int = np.zeros(5, dtype=int)
print("\nArray of zeros with integer type:")
print(zeros_int)  # Output: [0 0 0 0 0]
```



# Numpy – Zeros and ones

Numpy also provides other functions to create arrays with specific values.

The *full()* function creates an array of the given shape and fills it with the specified value.

The *identity()* function creates a square identity matrix (ones on the main diagonal and zeros elsewhere).

```
# Create an array filled with a specific value
full_array = np.full((2, 3), 7)
"""
[[7 7 7]
 [7 7 7]]
"""

# Create a 3x3 identity matrix
id_matrix = np.identity(3)
"""
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
"""
```

# Numpy – Zeros and ones

The *eye()* function creates a matrix with ones on the specified diagonal and zeros elsewhere.

```
# Alternative way to create identity matrix
id_matrix_alt = np.eye(3)
"""
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
"""

# Create a matrix with ones on the first
# diagonal above the main diagonal
eye_offset = np.eye(3, k=1)
"""
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 0.]]
"""
```



# Numpy – arange

The *arange()* function creates an array with evenly spaced values within a given interval.

It is similar to Python's built-in *range()* function but returns a Numpy array.

```
# Create array from 0 to 4
arr1 = np.arange(5)
print(arr1)  # Output: [0 1 2 3 4]

# Create array from 2 to 9
arr2 = np.arange(2, 10)
print(arr2)  # Output: [2 3 4 5 6 7 8 9]

# Create array from 1 to 9 with step 2
arr3 = np.arange(1, 10, 2)
print(arr3)  # Output: [1 3 5 7 9]

# Negative step
arr4 = np.arange(10, 0, -1)
print(arr4)  # Output: [10 9 8 7 6 5 4 3 2 1]

# Using float step
arr5 = np.arange(0, 2, 0.3)
print(arr5)  # Output: [0.  0.3 0.6 0.9 1.2 1.5 1.8]
```

# Numpy – linspace

The *linspace()* function creates an array with evenly spaced values over a specified interval. Unlike arange(), linspace() includes the end-point by default and specifies the number of elements rather than the step size.

*numpy.linspace(start, stop, num=50, endpoint=True)*

```
# Create array with 5 evenly spaced values from 0 to 1
arr1 = np.linspace(0, 1, 5)
print(arr1)  # Output: [0.   0.25 0.5  0.75 1.  ]

# Create array with 10 values from 0 to 2π
x = np.linspace(0, 2 * np.pi, 10)
print(x)  # Output: [0.         0.6981317  1.3962634
          #         2.0943951  2.7925268  3.4906585
          #         4.1887902  4.88692191 5.58505361
          #         6.28318531]

# You can get the step size along with the array
arr3, step = np.linspace(2, 3, 5, retstep=True)
print(f"Array: {arr3}")
print(f"Step size: {step}")
"""
Array: [2.   2.25 2.5  2.75 3.  ]
Step size: 0.25
"""
```



# Numpy – data types

Numpy's ndarrays are efficient because their elements must have the same type.

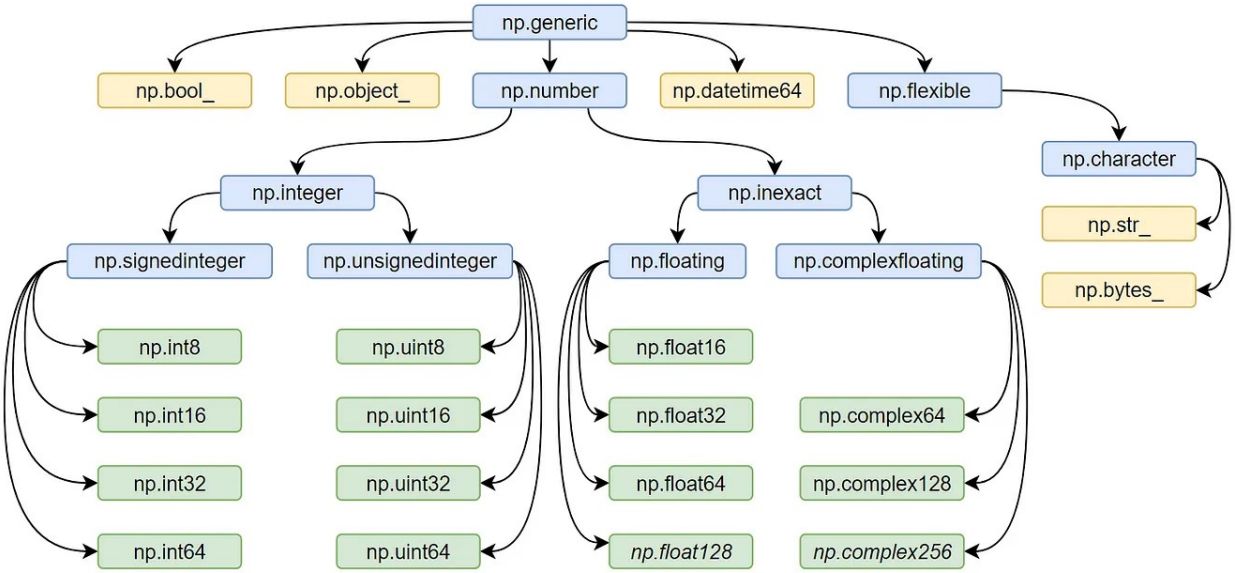
Numpy numerical types are instances of *numpy.dtype* objects

Data types: https://numpy.org/doc/stable/user/basics.types.html

Can be specified with the parameter *dtype* useful to avoid useless conversions.

```
z = np.arange(3, dtype=np.uint8)
# array([0, 1, 2], dtype=uint8)

# Can also be referred by code
z = np.array([1, 2, 3], dtype='f')
# array([1.,  2.,  3.], dtype=float32)

# Use function astype to convert
z.astype(np.float64)
# array([0.,  1.,  2.])
```

# Numpy – data types






# Numpy – random

The *numpy.random* module provides a comprehensive set of random number generation functions, with typical statistical distributions and directly in ndarray.

```
# Set random seed for reproducibility
np.random.seed(42)

# Generate a single random float between 0 and 1
rand_float = np.random.rand()
# 0.37454011884736246

# Generate an array of random floats between 0 and 1
rand_array = np.random.rand(3, 2)
# [[0.95071431 0.73199394]
#  [0.59865848 0.15601864]
#  [0.15599452 0.05808361]]

# Generate random integers from a range
rand_ints = np.random.randint(1, 100, size=(2, 3))
# [[95 96 32]
#  [35 39 87]]
```

# Numpy – random

```
# Generate random samples from a normal (Gaussian) distribution
gaussian_samples = np.random.normal(loc=0, scale=1, size=5)
print("\nSamples from normal distribution (mean=0, std=1):")
print(gaussian_samples)

# Randomly shuffle an array
arr = np.arange(10)
np.random.shuffle(arr)
print("\nShuffled array:")
print(arr)
```



# Numpy – efficiency

We can measure the difference between using Numpy and standard list operations

```
n = 10000
a = np.ones(n)
b = np.ones(n)
```

```
c = np.zeros(n)
timer = Timer()
for i in range(n):
    c[i] = a[i] + b[i]
f'{timer.stop():.5f} sec'
```

```
'0.01340 sec'
```

```
timer.start()
d = a + b
f'{timer.stop():.5f} sec'
```

```
'0.00019 sec'
```

```
import numpy as np
import time

class Timer:
    """Record multiple running times."""
    def __init__(self):
        self.times = []
        self.start()

    def start(self):
        """Start the timer."""
        self.tik = time.time()

    def stop(self):
        """Stop the timer and record the time in a list."""
        self.times.append(time.time() - self.tik)
        return self.times[-1]
```

# Numpy – saving and loading

Arrays can be saved to disk and loaded back. Numpy uses a .npy binary format that depends on the version of numpy.

I.e. It is not guaranteed that loading will work with different versions.

```
# Create an array
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Save to binary file (.npy format)
np.save('array_data.npy', arr)

# Load the array
loaded_arr = np.load('array_data.npy')
print(loaded_arr)
"""
[[1 2 3]
 [4 5 6]]
"""
```



# Numpy – saving and loading

Multiple arrays can also be saved and loaded into a .npz binary file.

```
# Save multiple arrays to a single file (.npz format)
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
np.savez('multiple_arrays.npz', a=arr1, b=arr2)

# Load multiple arrays
loaded_data = np.load('multiple_arrays.npz')
print(loaded_data['a'])  # [1 2 3]
print(loaded_data['b'])  # [4 5 6]
```

# Numpy – simple csv handling

Numpy can also use text files to save and load arrays.

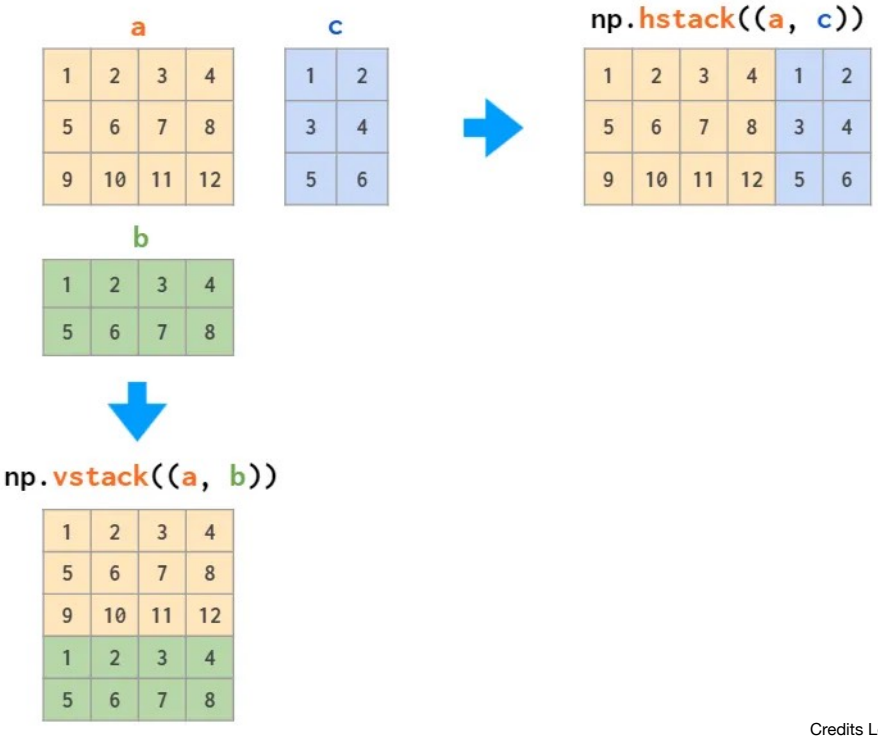
It can thus be used to handle simple csv files

```
# Create an array
arr = np.array([[1, 2, 3], [4, 5, 6]])

# Save to text file
np.savetxt('array_data.csv', arr, delimiter=',')

# Load from text file
loaded_arr = np.loadtxt('array_data.csv', delimiter=',')
print(loaded_arr)
"""
[[1. 2. 3.]
 [4. 5. 6.]]
"""
```



# Matrix manipulations

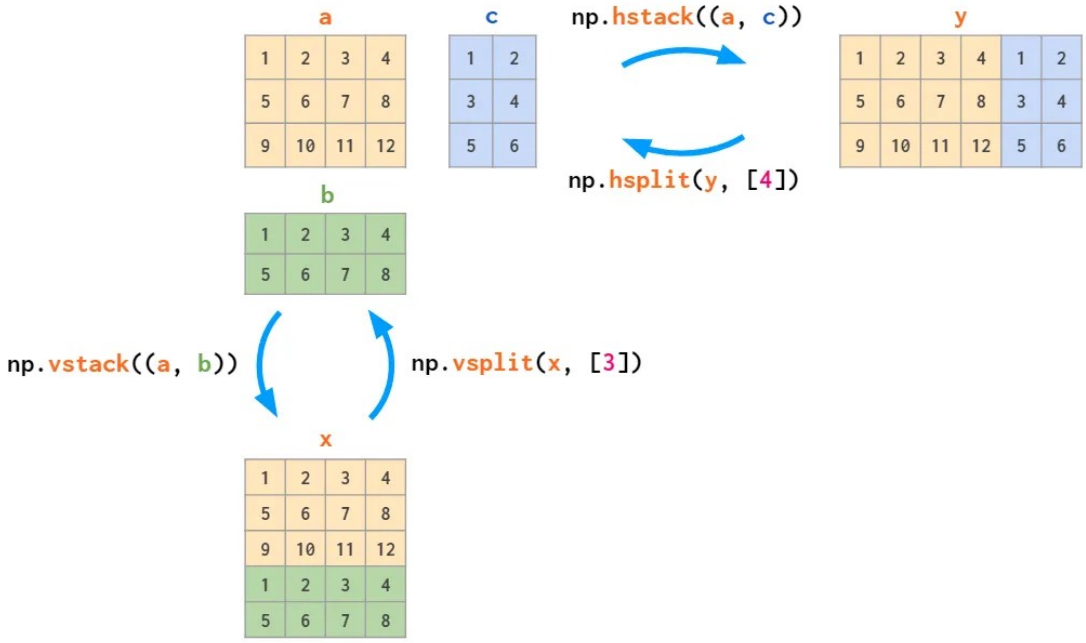
Numpy can join matrices with the [v/h] stack methods.




Credits Lev Maximov

# Matrix manipulations

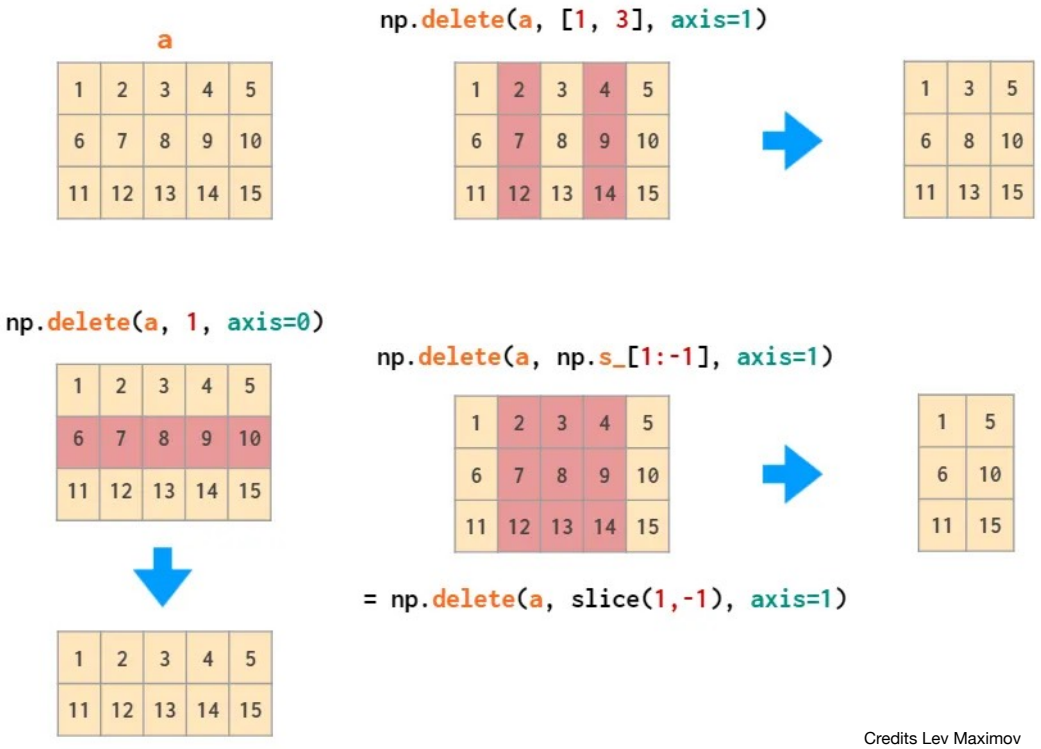
The opposite is splitting matrices






# Matrix manipulations

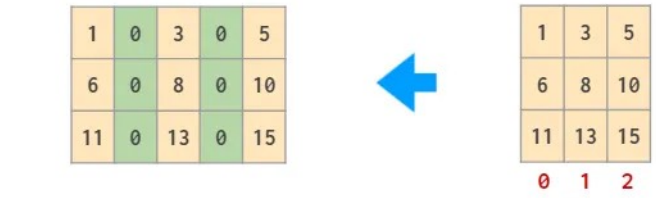
Rows and columns can be deleted



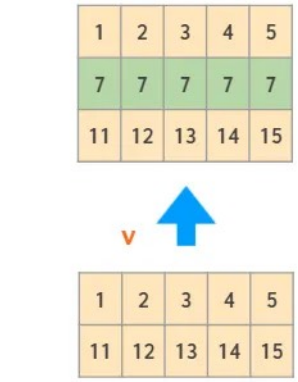
# Matrix manipulations

And also can be added with insert

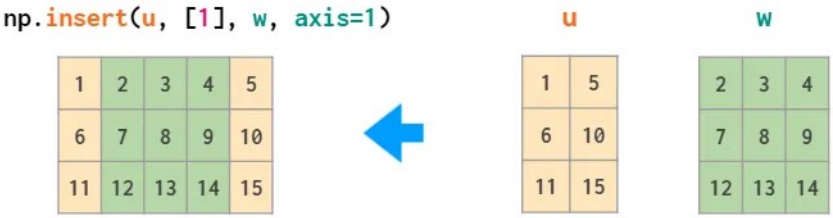
```
np.insert(h, [1, 2], 0, axis=1)
```




```
np.insert(v, 1, 7, axis=0)
```




```
np.insert(u, [1], w, axis=1)
```






# Numpy – Other

Numpy comes with many other useful features:

- Sorting operations
- Concatenation
- Searching
- Etc.

# Numpy – Exercises

- Create a 10x10 array with random values and find the minimum and maximum values.
- Create a 5x5 matrix with values 1,2,3,4 just below the diagonal.
  *hint: use the *diag* function
- Create a 10x10 array with random values and sort each row.
- Create a 2D array with 1 on the border and 0 inside.



## *NUMPY - ALGEBRA*

# Norm and dot product

Probably the most important basic functions for machine learning applications.

The *norm* of a vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \|\mathbf{x}\| = \sqrt{\sum_{i=1}^{n} x_i^2}$$

```
import numpy as np
x = np.array([3,4])

def vector_norm(vector):
    squares = [element**2 for element in vector]
    return sum(squares)**0.5

print("||", x, "|| =")
vector_norm(x)
```



# Norm and dot product

*Dot product* or *inner product* of two vectors

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^{n} x_i y_i$$

```
y = np.array([2,3])
```

```
np.dot(x,y)
```

```
18
```

```
x.dot(y)
```

```
18
```

# Matrix multiplication

Let us consider the following form

$$x_1 \begin{bmatrix} a_{11} \\ \vdots \\ a_{m1} \end{bmatrix} + \ldots + x_n \begin{bmatrix} a_{1n} \\ \vdots \\ a_{mn} \end{bmatrix} = \begin{bmatrix} a_{11} & \ldots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \ldots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

```
A = np.array([
    [1,2,3],
    [4,5,6]])
A
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

**Means everything else**

```
x = np.array([2,4,5]).reshape(-1,1)
x
```

```
array([[2],
       [4],
       [5]])
```

```
np.matmul(A,x) # matrix multiplication
```

```
array([[25],
       [58]])
```



# Matrix multiplication

Also works for the generic form of M1 * M2 where the two matrices have neighbouring dimensions match

$$\underbrace{\mathbf{A}}_{n \times k} \quad \underbrace{\mathbf{B}}_{k \times m} = \mathbf{C}$$

```
# Matrix multiplication
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])
matrix_product = np.matmul(m1, m2)
print(matrix_product)
"""
[[19 22]
 [43 50]]
"""

# Alternative matrix multiplication notation
# (python 3.5+)
matrix_product_alt = m1 @ m2
```

# Transpose

To transpose a matrix, just use the .T property

For higher dimensionality, use the *transpose()* function which accepts a new arrangement of dimensions.

```
# Transpose the array
transposed = arr.T
print(transposed)
"""
[[1 4]
 [2 5]
 [3 6]]
"""

# For higher dimensions
arr_3d = np.arange(24).reshape(2, 3, 4)
# Transpose axes in different orders
transposed_3d = np.transpose(arr_3d, (1, 0, 2))
```



# Matrix inverse

The *linalg* sub-package contains several functions dedicated to computing basic linear algebra algorithms, such as the matrix inverse:

```
A = np.array([[1, 2], [3, 4]])

# Matrix inverse
inv_A = np.linalg.inv(A)
print("Inverse of A:")
print(inv_A)
"""
[[-2.   1. ]
 [ 1.5 -0.5]]
"""
```

# Notable linear algebra functions

```
# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:")
print(eigenvectors)

# Solve linear system Ax = b
b = np.array([1, 2])
x = np.linalg.solve(A, b)
print("Solution to Ax = b:", x)
```



# Statistical functions

Numpy supports all the basic statistical functions such as mean, median, etc.

```
# Create an array
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Basic statistics
print(f"Mean: {np.mean(arr)}")
print(f"Median: {np.median(arr)}")
print(f"Standard deviation: {np.std(arr)}")
print(f"Variance: {np.var(arr)}")
print(f"Min: {np.min(arr)}, Max: {np.max(arr)}")

# Percentiles
print(f"25th percentile: {np.percentile(arr, 25)}")
print(f"75th percentile: {np.percentile(arr, 75)}")

# For 2D arrays
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Mean along rows:", np.mean(arr_2d, axis=1))
print("Mean along columns:", np.mean(arr_2d, axis=0))
```

# Exercises

- Create two vectors and calculate their dot product, norms, and the angle between them.

$$\cos(\Theta) = (u \cdot v)/(||u|| \cdot ||v||)$$

- Create a 3x3 matrix and find its eigenvalues and eigenvectors. Verify that Av = λv for each eigenvalue-eigenvector pair

- Solve a system of linear equations Ax = b using Numpy.
  Verify your solution by computing Ax.
  A = np.array([[3, 1, -1], [1, 4, 1], [2, 1, 2]])
  b = np.array([2, 12, 10])



# Exercises

- Perform statistical analysis on a dataset using NumPy functions.

```
# Create a 2D array representing exam scores for different subjects
scores = np.array([
    [85, 90, 78, 92, 88],  # Math
    [75, 82, 95, 79, 88],  # Science
    [92, 86, 80, 91, 84]   # English
])
```

- Find the average score for each subject
- Find the student with the highest average score across all subjects
  *Hint: the *argmax* function may be helpful
- Calculate the standard deviation for each subject
- Determine the correlation between math and science scores
  *Hint: the corrcoef function can compute Pearson correlation between two arrays